【扔物线 Compose 课程讲义】3.2-MutableState 和 mutableStateOf()

写法

Compose 界面刷新的三个阶段

mutableStateOf()

Kotlin 的 by 和 by mutableStateOf() 的 by

关于讲义

讲义是对课程内容中**所有结论的集中汇总**。所以在看完每节课程之后，迅速来看一遍讲义，对课程内容做回顾，会有非常好的吸收效果。

面试前不要背讲义

在面试之前，通过快速浏览一遍讲义来确认所有知识都没有忘记，是个很好的备战方式。

但**不要用【背讲义上的知识点】的方式来备战**。最有效的浏览方式是：

浏览讲义中的所有内容，并：

- 看到「懂，并且记得背后的原因（或技术细节）」的，直接跳过；
- 看到「懂，但细节已经忘了」的，快速回看当节课程，把知识捡起来；
- 看到「已经不太懂 / 完全忘了」的，认真重看当街课程，把知识学回来。

疑问或意见

如果你发现讲义或课程有错误，或有别的什么意见，可以向任何一个你认识的助教反馈。

如果认可我们的课程，还请给个好评！ 评价链接：rengwuxian.com/cp

写法

```kotlin
var name by mutableStateOf("rengwuxian")
```

Compose 界面刷新的三个阶段

- Compose 阶段（Composition 阶段、组合阶段）：执行 `@Composable` 代码，并创造出实际的界面元素；
  - 是 Compose 比传统 View 系统多的阶段。
- 布局阶段：用组合阶段创造出的界面元素来进行测量和布局；
- 绘制阶段：用布局阶段测量完成的界面元素来进行绘制。

`mutableStateOf()`

- 返回值类型：`MutableState`
- 最终创建的对象类型：`ParcelableSnapshotMutableState`
- 关键代码的位置：`SnapshotMutableStateImpl`
  - `value.get()`：获取这个 `StateObject` 最新的有效 `StateRecord` 值后返回它的 `value`，并记录下读的位置；
  - `value.set()`：获取或创建这个 `StateObject` 中的、对应当前 `Snapshot` 的 `id` 的 `StateRecord`，赋值给它的 `value`，然后把读过这个 `StateObject` 的地方，把 Recompose Scope 标记为失效（等待下一帧的 Recompose。
- 完整的订阅和自动更新机制：
  - 一是上面的 `SnapshotMutableStateImpl`（即 `MutableState` 的实现类）的 `value` 的 `get()` 和 `set()` 函数所包含的「记录读」与「写时标记 Recompose Scope 失效」；
  - 另外还有一个重点：在 `StateObject` 中的新值应用的时候，也进行标记失效的操作（后面的课程里会有细节讲解）。

Kotlin 的 `by` 和 `by mutableStateOf()` 的 by

- Kotlin 的 `by`：
  - Kotlin 的原生关键字，`by` 的右边是代理对象
  - 代理对象的 `getValue()` 和 `setValue()` 都有严格的要求，符合要求就可以进行 `get()` 和 `set()` 函数的代理
- `by mutableStateOf()` 的 `by`：
  - 首先，它是一个 Kotlin 的 `by`；

- 然后，`State` 被添加了扩展函数 `getValue()`，并且 `MutableState` 被添加了扩展函数 `setValue()`，这两个函数符合 Kotlin 的 `by` 的要求，因此 `by mutableStateOf()` 是可用的。